

1 Deep Learning Lab Assignment - NLP

Lab Title: NLP Techniques for Text Classification

Course: Deep Learning

Student Name: Md Yaseen Alam

Student ID: 202201040060 **Group**

Members:

1. Vaibhav Satish Jadhav
2. Vidya Bingi
3. Om Borle

1.1 Objective

The objective of this assignment is to implement NLP preprocessing techniques and build a text classification model using machine learning techniques.

1.2 Dataset Link: <https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv>

1.3 GitHub Repository Link: <https://github.com/Dante-hero/Lab-Assignment-No.-4--NLP-Preprocessing-And-Text-Classification>

1.4 Colab Link: <https://colab.research.google.com/drive/1e1PdmGeO-OinzWGtd64gn15bY6G6kPMf?usp=sharing>

Learning Outcomes:

1. Understand and apply NLP preprocessing techniques such as tokenization, stopword removal, stemming, and lemmatization.
2. Implement text vectorization techniques such as TF-IDF and CountVectorizer.
3. Develop a text classification model using a machine learning algorithm.
4. Evaluate the performance of the model using suitable metrics.

2 Assignment Instructions:

Part 1: NLP Preprocessing Dataset

Selection:

Choose any text dataset from **Best Datasets for Text** <https://en.innovatiana.com/post/bestdatasets-for-text-classification>, such as SMS Spam Collection, IMDb Reviews, or any other relevant dataset.

Download the dataset and upload it to Google Colab.

Load the dataset into a Pandas DataFrame and explore its structure (e.g., check missing values, data types, and label distribution).

Text Preprocessing:

Convert text to lowercase.

Perform tokenization using NLTK or spaCy.

Remove stopwords using NLTK or spaCy.

Apply stemming using PorterStemmer or SnowballStemmer.

Apply lemmatization using WordNetLemmatizer.

Vectorization Techniques:

Convert text data into numerical format using TF-IDF and CountVectorizer.

```
[6]: # Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, SnowballStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, roc_curve, auc,
    precision_recall_curve, classification_report
)
import string

# Download NLTK data
nltk.download('punkt', force=True)
nltk.download('punkt_tab')
nltk.download('stopwords', force=True)
nltk.download('wordnet', force=True)
nltk.download('omw-1.4', force=True)
```

```
[nltk_data] Downloading package punkt to
/root/nltk_data... [nltk_data] Unzipping
tokenizers/punkt.zip.
```

```
[nltk_data] Downloading package punkt_tab to
/root/nltk_data... [nltk_data] Package punkt_tab is
already up-to-date! [nltk_data] Downloading package
stopwords to /root/nltk_data... [nltk_data] Unzipping
corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to
/root/nltk_data...
[nltk_data] Downloading package omw-1.4 to
/root/nltk_data...
```

```
[6]: True
```

```
[7]: # Load dataset df =
pd.read_csv("https://raw.githubusercontent.com/justmark
ham/
pycon-2016-tutorial/master/data/sms.tsv", sep='\t',
header=None, _
names=['label', 'message'])
df.head()
```

```
[7]: label message
0 ham Go until jurong point, crazy.. Available only ...
1 hamOk lar... Joking wif u oni...
2 spam Free entry in 2 a wkly comp to win FA Cup fina...
3 ham U dun say so early hor... U c already then say...
4 ham Nah I don't think he goes to usf, he lives aro...
```

```
[8]: stop_words = set(stopwords.words("english"))
stemmer = SnowballStemmer("english")
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    text = text.lower()
    tokens = nltk.word_tokenize(text)
    tokens = [word for word in tokens if word.isalpha() and word not in _
stop_words]
    stemmed = [stemmer.stem(word) for word in tokens]
    lemmatized = [lemmatizer.lemmatize(word) for word in stemmed]
    return " ".join(lemmatized)

df['clean_text'] = df['message'].apply(preprocess)
df.head()
```

```
[8]: label message \
0 ham Go until jurong point, crazy.. Available only ...
1 hamOk lar... Joking wif u oni...
2 spam Free entry in 2 a wkly comp to win FA Cup fina...
3 ham U dun say so early hor... U c already then say...
```

```

4    ham Nah I don't think he goes to usf, he lives aro...

                                clean_text
0      go jurong point crazi avail
      bugi n great world...
1      ok lar joke wif u oni
2      free entri wkli comp win fa
      cup final tkts may...
3      u dun say earli hor u c
      already say
4      nah think goe usf live around
      though

```

Splitting the Data:

Divide the dataset into training and testing sets (e.g., 80% training, 20% testing).

Building the Classification Model:

Train a text classification model using Logistic Regression, Naïve Bayes, or any other suitable algorithm.

Implement the model using scikit-learn.

Model Evaluation:

Evaluate the model using accuracy, precision, recall, and F1-score.

Use a confusion matrix to visualize the results.

```

[9]: # Vectorization
tfidf = TfidfVectorizer()
X = tfidf.fit_transform(df['clean_text'])
y = df['label'].map({'ham': 0, 'spam': 1})

# Splitting
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Model Training
model = MultinomialNB()
model.fit(X_train, y_train)

# Prediction & Evaluation
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))

```

```

print("F1 Score:", f1_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Plotting
plt.figure(figsize=(18, 5))

# Confusion Matrix
plt.subplot(1, 3, 1)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Ham', 'Spam'],
            yticklabels=['Ham', 'Spam'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')

# ROC Curve
plt.subplot(1, 3, 2)
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.grid(True)

# Precision-Recall Curve
plt.subplot(1, 3, 3)
precision, recall, _ = precision_recall_curve(y_test, y_prob)
plt.plot(recall, precision, marker='.')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.grid(True)

plt.tight_layout()
plt.show()

# Classification Report Heatmap
report = classification_report(y_test, y_pred, target_names=['Ham', 'Spam'],
                               output_dict=True)
sns.heatmap(pd.DataFrame(report).iloc[:-1, :].T, annot=True, cmap='YlGnBu')
plt.title('Classification Report')
plt.show()

```

Accuracy: 0.9650224215246637

Precision: 0.9824561403508771

Recall:

0.7516778523489933 F1

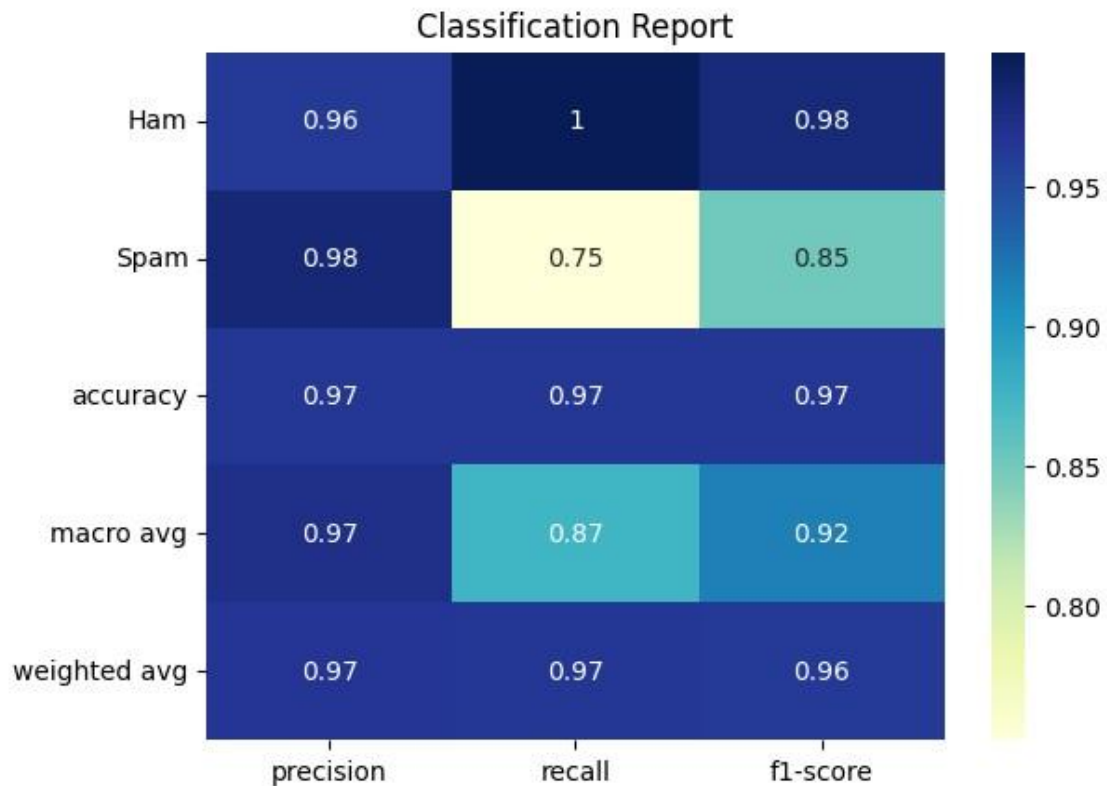
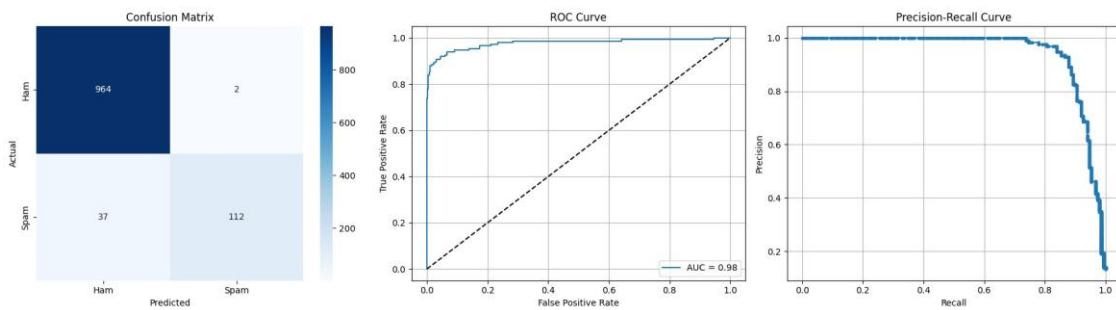
Score:

0.8517110266159695

Confusion Matrix:

```
[[964  2]
```

```
[ 37 112]]
```



Submission Guidelines:

Google Colab Notebook Submission:

Save your notebook as NLP_Text_Classification_YourName.ipynb.

Ensure all code cells are executed, and the output is visible.

Include proper documentation and comments explaining each step.

Report Submission (Optional):

Prepare a short report (2-3 pages) summarizing your approach, findings, and model performance.

Upload the report along with the Colab Notebook.

Grading Criteria:

Correct implementation of NLP preprocessing (30%)

Effective use of vectorization techniques (20%)

Model accuracy and performance evaluation (30%)

Code clarity, documentation, and presentation (20%)